Université Tunis El Manar
Faculté des Sciences Économiques et de Gestion de Tunis (FSEGT)
— AGL 2025/2026

RAPPORT DE PROJET — AGL

YinPulse

Restore. Rebuild. Renew.

Plateforme de Tourisme Médical & Bien-être

Membre	Rôle Scrum	Fonctionnalité	Classe Java
Chaima Arfaoui A04	Product Owner + Dev	Authentification & Inscription	`Utilisateur.java`
Farah Boukhatem A01	Scrum Master + Dev	Recherche & Filtrage Services	`Service.java`
Malek Homri A04	Développeur	Profil Médecin & Équipe	`Medecin.java`
Eya Sellini A01	Développeuse	Avis & Notation	`Avis.java`
Maram Jebeli A01	Développeuse	Tableau de Bord Admin	`Admin.java`

Encadrante : Mme Fatma Karray
Dépôt GitHub : https://github.com/chaimaarfaoui-ui/aglpro
9 classes Java | 7 scénarios complets | 2 Sprints | Avril 2026

Dépôt GitHub : https://github.com/chaimaarfaoui-ui/aglpro | 9 classes Java | 7 scénarios complets | 2 Sprints

1. Fiche Projet

Nom du projet

YinPulse — Plateforme de Tourisme Médical & Bien-être

YinPulse est une plateforme tunisienne qui met en relation des patients internationaux avec des chirurgiens spécialisés en reconstruction post-accident, chirurgie bariatrique et greffe capillaire, tout en proposant des retraites de convalescence (Hammamet, Aïn Draham, Tozeur). Le tout en un seul parcours guidé de 7 étapes : **Your Journey**.

Problème résolu & Cible

Un patient étranger souhaitant se faire opérer en Tunisie doit jongler entre trouver un chirurgien qualifié, organiser l'hébergement pré/post-op, et planifier sa convalescence — sans aucun accompagnement unifié. YinPulse centralise l'ensemble de ce parcours.

Cible : Patients européens et tunisiens de la diaspora | Chirurgiens spécialisés | Administrateurs de la plateforme.

Acteurs du système

Acteur	Description	Héritage
Visiteur	Consulte les services et profils médecins sans compte	—
Patient	S'inscrit, réserve des consultations, laisse des avis	hérite Visiteur
Médecin	Gère son profil, ses tarifs et ses disponibilités	—
Administrateur	Supervise la plateforme, modère les avis, gère les médecins	hérite Patient

Fonctionnalités principales (1 par membre)

Membre	Rôle	Fonctionnalité	User Stories
Chaima (PO)	Product Owner + Dev	Authentification & Inscription	US-01, US-02
Farah (SM)	Scrum Master + Dev	Recherche & Filtrage des Services	US-03, US-04
Malek	Développeur	Profil Médecin & Équipe	US-05, US-06
Eya	Développeur	Avis & Notation	US-07, US-08
Maram	Développeur	Tableau de Bord Admin	US-09, US-10

US-04	Priorité : Must	SP : 5	Responsable : Farah

En tant que Patient, je veux filtrer les services par catégorie, localisation et budget afin d'affiner ma recherche.

Critères d'acceptation
✓ Si localisation = null → critère ignoré (filtre optionnel)

2. Product Backlog

Le backlog complet contient 10 User Stories avec priorité MoSCoW, critères d'acceptation (minimum 3 par US) et estimation en Story Points (Fibonacci). Chaque membre a rédigé 2 US.

US-01	Priorité : Must	SP : 3	Responsable : Chaima
En tant que Visiteur, je veux m'inscrire avec email + mdp afin d'accéder aux services YinPulse.			

Critères d'acceptation
✓ Email valide : doit contenir '@' — null check inclus (email == null → [ERREUR])
✓ Mot de passe ≥ 6 caractères — message [ERREUR] si trop court
✓ Message [OK] affiché avec prénom + nom si inscription réussie
✓ Rôle PATIENT affecté automatiquement après inscription réussie
✓ dateInscription enregistrée automatiquement

US-02	Priorité : Must	SP : 2	Responsable : Chaima
En tant que Patient, je veux me connecter avec mes identifiants afin d'accéder à mon espace.			

Critères d'acceptation
✓ Connexion réussie si email ET mot de passe correspondent exactement
✓ Message [OK] 'Bienvenue Prénom Nom' affiché après connexion réussie
✓ Message [ERREUR] 'Email ou mot de passe incorrect' si identifiants erronés
✓ Attribut estConnecte = true après connexion réussie
✓ Méthode seDeconnecter() remet estConnecte = false

US-03	Priorité : Must	SP : 3	Responsable : Farah
En tant que Patient, je veux rechercher un service par mot-clé afin de trouver rapidement ce dont j'ai besoin.			

Critères d'acceptation
✓ La recherche est insensible à la casse (toLowerCase() appliqué)
✓ La recherche couvre : nom, catégorie, description ET localisation du service
✓ Retourne true si le mot-clé est trouvé dans au moins un des 4 champs

✓ Retourne false si motCle est null ou vide

✓ Résultats affichés avec nom + prix en DTé

US-08	Priorité : Must	SP : 2	Responsable : Eya

En tant qu'Admin, je veux valider ou rejeter un avis en attente afin de modérer la qualité du contenu.

Critères d'acceptation

✓ valider() → valide = true + message [OK] 'Avis #X de Y validé'

✓ Si catégorie = null → critère ignoré (filtre optionnel)

✓ Si prixMax <= 0 → critère ignoré (correction bug : prixMax == 0 insuffisant)

✓ Seuls les services disponible = true sont retournés

✓ Filtrage combiné : tous les critères non-null doivent être satisfaits simultanément

US-05	Priorité : Must	SP : 2	Responsable : Malek

En tant que Patient, je veux consulter le profil complet d'un médecin afin de choisir en confiance.

Critères d'acceptation

✓ Affichage : nom, prénom, spécialité, cabinet, gouvernorat

✓ Affichage : tarif en DT, années d'expérience, diplôme

✓ Affichage : note moyenne /5, nombre de consultations, statut actif/inactif

✓ Contact affiché : téléphone + email

✓ Format lisible en console avec indentation cohérente

US-06	Priorité : Should	SP : 2	Responsable : Malek

En tant que Médecin, je veux mettre à jour mon tarif et mon adresse afin que mes informations soient exactes.

Critères d'acceptation

✓ Paramètre null ou vide → champ non modifié (mise à jour partielle autorisée)

✓ Nouveau tarif <= 0 → tarif non modifié

✓ Message [OK] 'Profil du Dr. X mis à jour' affiché après modification réussie

✓ Les anciens champs non fournis restent inchangés

✓ Mise à jour appliquée immédiatement sur l'objet en mémoire

US-07	Priorité : Must	SP : 3	Responsable : Eya

En tant que Patient, je veux soumettre un avis après consultation afin d'aider les futurs patients.

Critères d'acceptation

✓ Note doit être entre 1 et 5 inclus — [ERREUR] si hors plage

✓ Commentaire ≥ 10 caractères APRÈS trim() — espaces seuls refusés

✓ Statut initial : valide = false (en attente de validation admin)

✓ Message [OK] 'Avis soumis : X/5 - en attente de validation' si réussi

✓ dateAvis enregistrée automatiquement à la soumission

✓ rejeter(motif) → valide = false + reponseAdmin = motif

✓ Compteur nombreAvisEnAttente décrémenté après validation ou rejet

✓ Guard : compteur ne descend jamais en dessous de 0

✓ repondre(reponse) permet d'ajouter une réponse admin à l'avis

US-09	Priorité : Must	SP : 5	Responsable : Maram

En tant qu'Admin, je veux afficher un tableau de bord avec statistiques clés afin de superviser la plateforme.

Critères d'acceptation

✓ Affichage : nombre d'utilisateurs inscrits, médecins actifs, services disponibles

✓ Affichage : nombre d'avis en attente + revenus du jour en DT

✓ Alerte [ALERTE] affichée si nombreAvisEnAttente > 0

✓ Nom de l'admin + niveau d'accès + date de dernière connexion affichés

✓ Format tableau de bord lisible avec séparateurs visuels

US-10	Priorité : Must	SP : 3	Responsable : Maram

En tant qu'Admin, je veux ajouter ou désactiver un médecin afin de gérer le réseau de praticiens.

Critères d'acceptation

✓ ajouterMedecin(medecin) : null check — [ERREUR] si médecin invalide

✓ ajouterMedecin() incrémente nombreMedecins et affiche message [OK]

✓ supprimerMedecin() appelle medecin.desactiver() puis décrémente nombreMedecins

✓ desactiver() positionne actif = false sur l'objet Medecin

✓ Guard : médecin null ignoré silencieusement dans supprimerMedecin()

Vélocité totale Sprint 1 : 30 Story Points (US-01 à US-10).

Sprint 2 : 4 classes complémentaires (Retraite, Hotel, Consultation, Parcours) + logique métier complète + 7 scénarios.

SPRINT 1 — « Les fondations »

Sprint 1 Goal

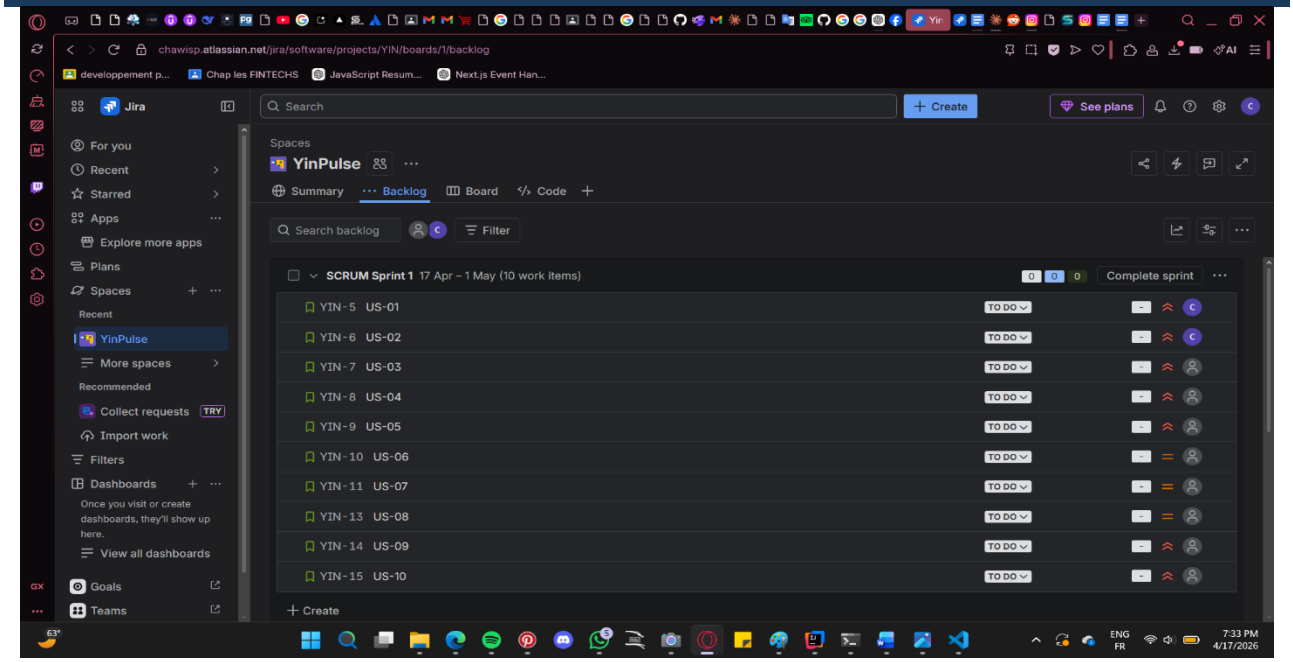
Concevoir l'architecture Java exécutable du système et livrer les 5 classes principales avec leur logique de base. Chaque membre intègre sa classe via Pull Request. Livrer les maquettes Figma navigables pour validation PO.

Membre	Tâche technique	US	Statut
Chaima	Créer Utilisateur.java — sInscrire(), seConnecter(), validation email/mdp	US-01, 02	■ Done
Farah	Créer Service.java — correspondARecherche(), filtrerCombine()	US-03, 04	■ Done
Malek	Créer Medecin.java — afficherProfil(), mettreAJourNote()	US-05, 06	■ Done
Eya	Créer Avis.java — soumettre() avec trim(), valider(), rejeter()	US-07, 08	■ Done
Maram	Créer Admin.java extends Utilisateur — tableau de bord, gestion médecins	US-09, 10	■ Done
Tous	Créer Consultation.java (classe commune partagée)	—	■ Done
Chaima	Configurer dépôt GitHub, créer 5 branches, ouvrir 5 PRs	—	■ Done
Tous	Maquettes Figma — 2 écrans par membre (10 écrans total)	—	■ Done

Tableau Kanban — Sprint 1

■ À faire	■ En cours	■ En revue	■ Terminé
Créer dépôt GitHub Branches features Maquettes Figma	Utilisateur.java Service.java	Medecin.java Avis.java Admin.java	Consultation.java Main.java V1 Pull Requests x5

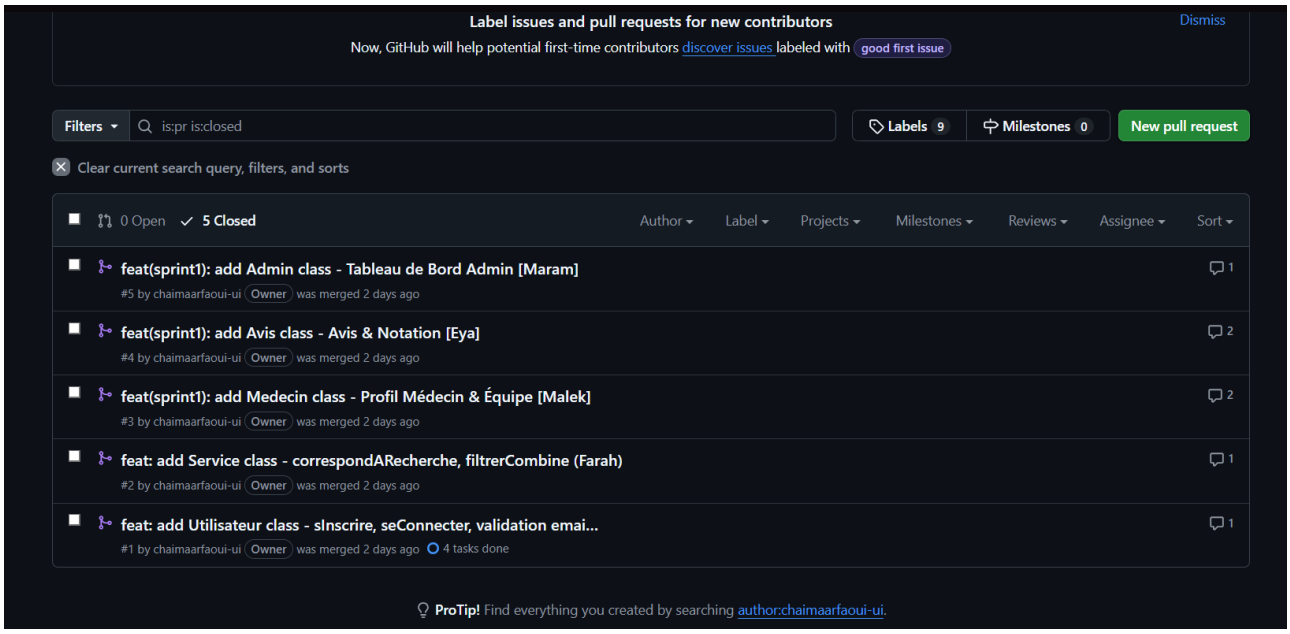
■ *Captures d'écran Kanban disponibles dans docs/captures/kanban/ du dépôt GitHub.*



Pull Requests — Sprint 1

5 Pull Requests ont été ouvertes sur GitHub (une par membre), reviewées et mergées dans `main`. Chaque PR a été approuvée par au moins un autre membre. C'est lors de ces reviews que les 3 bugs critiques ont été détectés et corrigés.

PR	Branche → main	Reviewer	Bug détecté	Statut
PR #1 — Chaima	`feature/authentification → main`	Farah	`this.role manquant`	✓ Mergée
PR #2 — Farah	`feature/recherche-services → main`	Chaima	`prixMax == 0 → prixMax <= 0`	✓ Mergée
PR #3 — Malek	`feature/profil-medecin → main`	Eya	`div/0 si nombreConsultations == 0`	✓ Mergée
PR #4 — Eya	`feature/notation-avis → main`	Maram	`trim() absent → espaces valides`	✓ Mergée
PR #5 — Maram	`feature/admin-dashboard → main`	Chaima	`super() vide → params perdus`	✓ Mergée



Captures d'écran des PRs disponibles dans docs/captures/reviews/ du dépôt GitHub : https://github.com/chaimaarfaoui-ui/aglpro

Definition of Done — Sprint 1

- ■ Le code compile sans erreur
- ■ Le Main.java exécute le scénario de base de chaque fonctionnalité
- ■ Chaque classe intégrée sur main via Pull Request reviewée
- ■ Au moins 1 collègue a relu et commenté chaque PR

SPRINT 2 — « Le produit complet »

Sprint 2 Goal

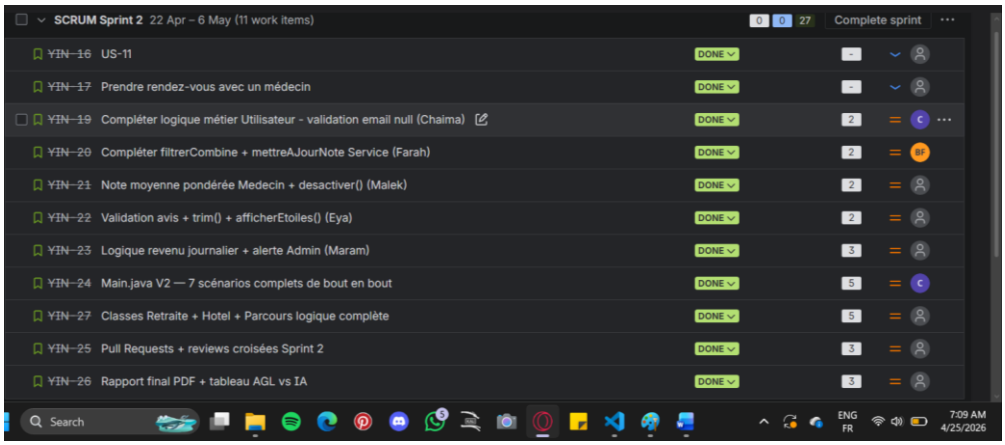
Livrer un prototype complet avec logique métier réelle (9 classes, 7 scénarios de bout en bout). Rapport consolidé et soutenance préparée.

Membre	Tâche technique	Statut
Farah	Ajouter Retraite.java — calculerPrixSejour(), recommanderPackage(), filtrage env/budget	■ Done
Malek	Ajouter Hotel.java — filtrerCombine(), reserver(), estAdaptePostOperatoire()	■ Done
Chaima	Ajouter Parcours.java — 7 étapes, avancerEtape(), calculerProgression(), ajouterCout()	■ Done
Tous	Main.java V2 — 7 scénarios complets (Inscription → Admin)	■ Done
Tous	2e diagramme de séquence (parcours complet Sprint 2)	■ Done
Tous	Rapport final + soutenance	■ Done

Tableau Kanban — Sprint 2

■ À faire	■ En cours	■ En revue	■ Terminé

Rapport final Soutenance slides	Retraite.java Hotel.java Parcours.java	Main.java V2 Diagramme DS Sprint 2	9 classes Java 7 scénarios Rapport PDF



Vélocité — comparaison prévue vs réalisée

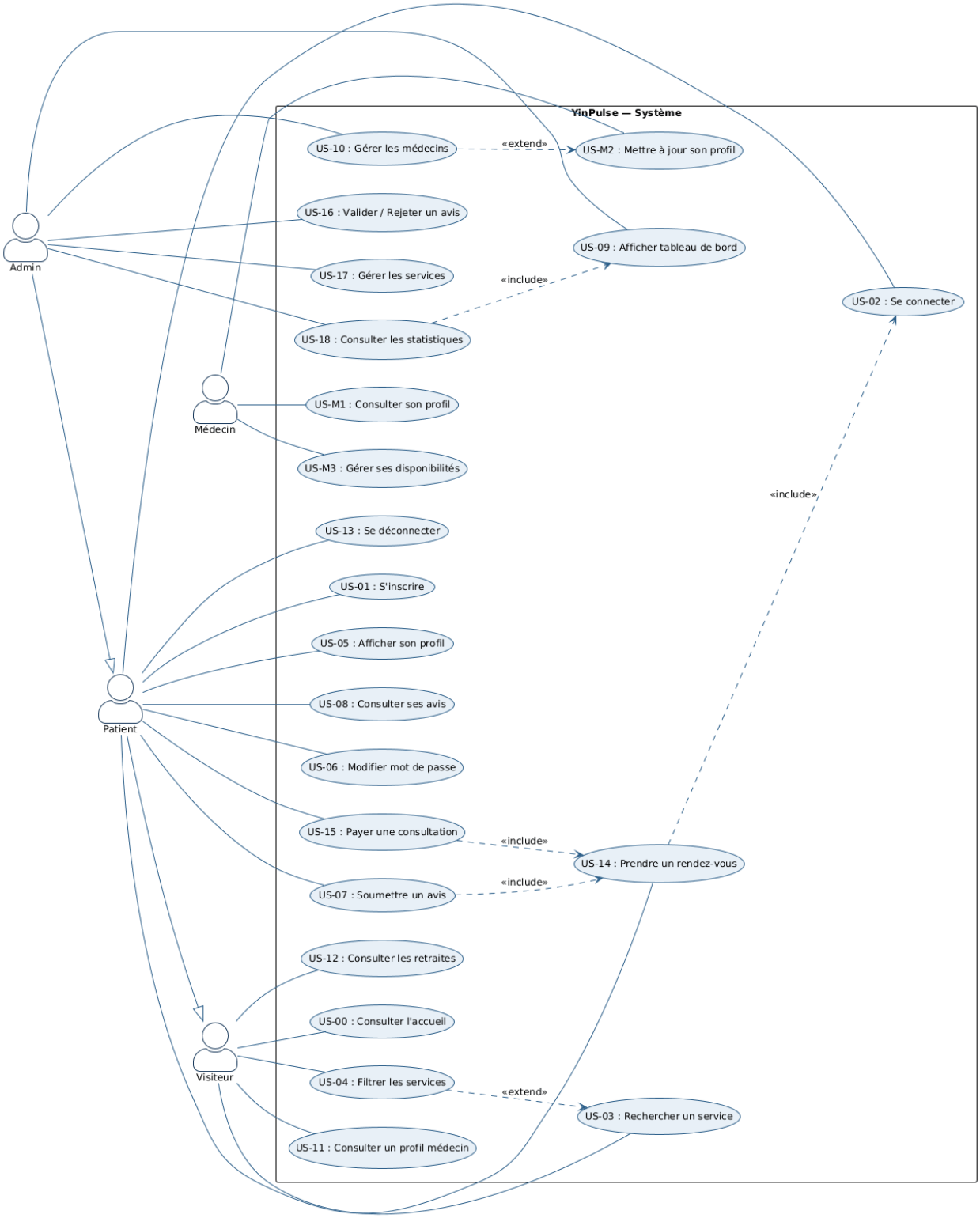
Sprint	US prévues	SP prévus	US livrées	SP réalisés	Verdict
Sprint 1	10 US	30 SP	10 US	30 SP	■ 100%
Sprint 2	4 classes +	~20 SP	4 classes +	~20 SP	■ 100%

4.1 Diagramme de Cas d'Utilisation (DCU)

Réalisé avec PlantUML. Le DCU couvre 4 acteurs et 18 cas d'utilisation avec relations d'héritage, <<include>> et <<extend>>. Fichier : **diagrammes/DCU_YinPulse.puml**

Acteur	Cas d'utilisation principaux
Visiteur	UC00 Consulter accueil | UC03 Rechercher service | UC04 Filtrer | UC11 Profil médecin | UC12 Retraites
Patient (hérite Visiteur)	UC01 S'inscrire | UC02 Se connecter | UC13 Se déconnecter | UC05 Profil | UC14 RDV | UC07 Soumettre avis
Médecin	UCM1 Consulter son profil | UCM2 Mettre à jour profil | UCM3 Gérer disponibilités
Admin (hérite Patient)	UC09 Tableau de bord | UC10 Gérer médecins | UC16 Valider/Rejeter avis | UC17 Gérer services | UC18 Statistiques

Diagramme de Cas d'Utilisation — YinPulse
Plateforme de Tourisme Médical & Bien-être

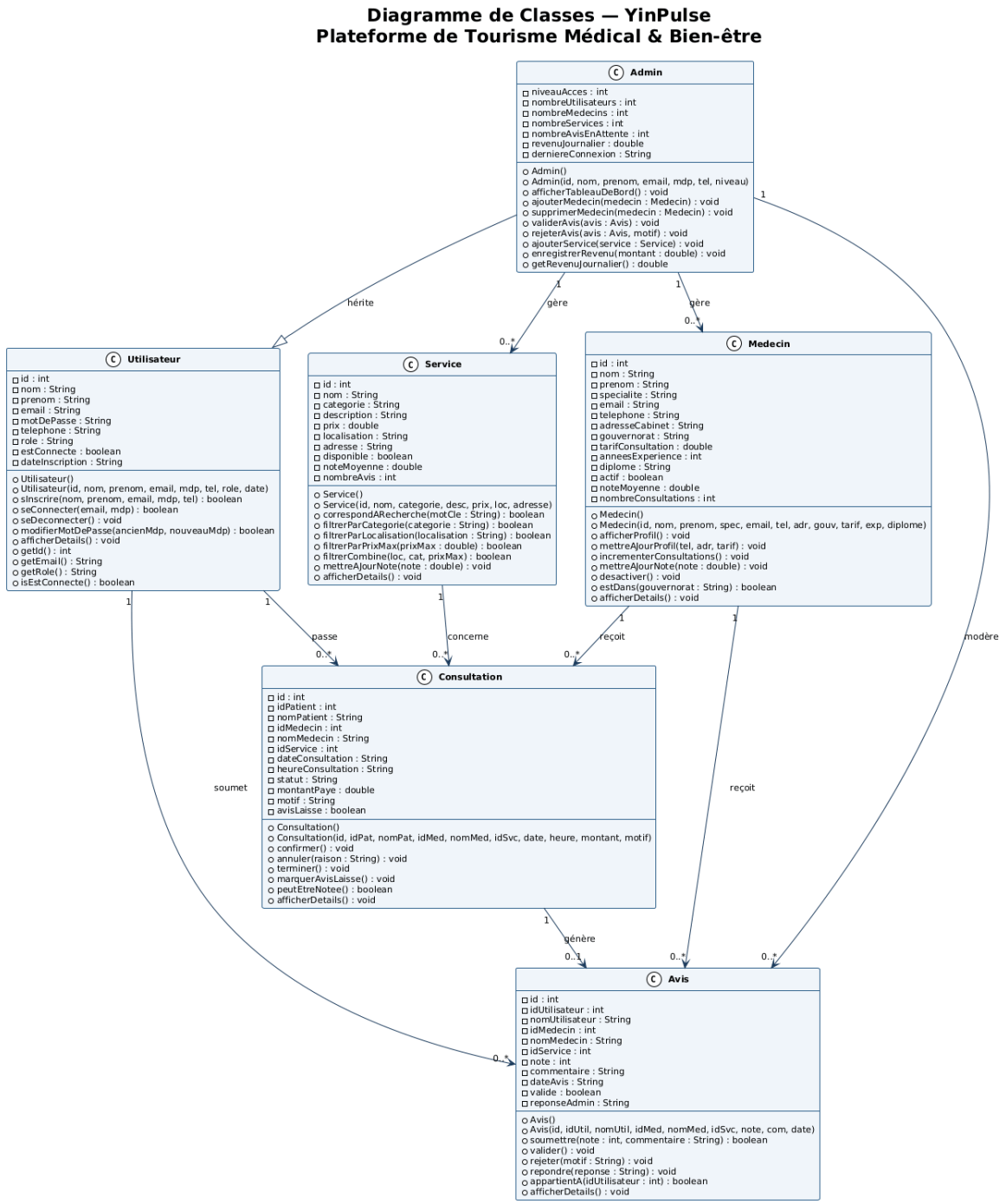


4.2 Diagramme de Classes (DC) — 9 classes

Architecture objet complète Sprint 2. **9 classes Java** avec champs privés, constructeurs, méthodes métier et getters/setters. Fichier : **diagrammes/DC_YinPulse.puml**

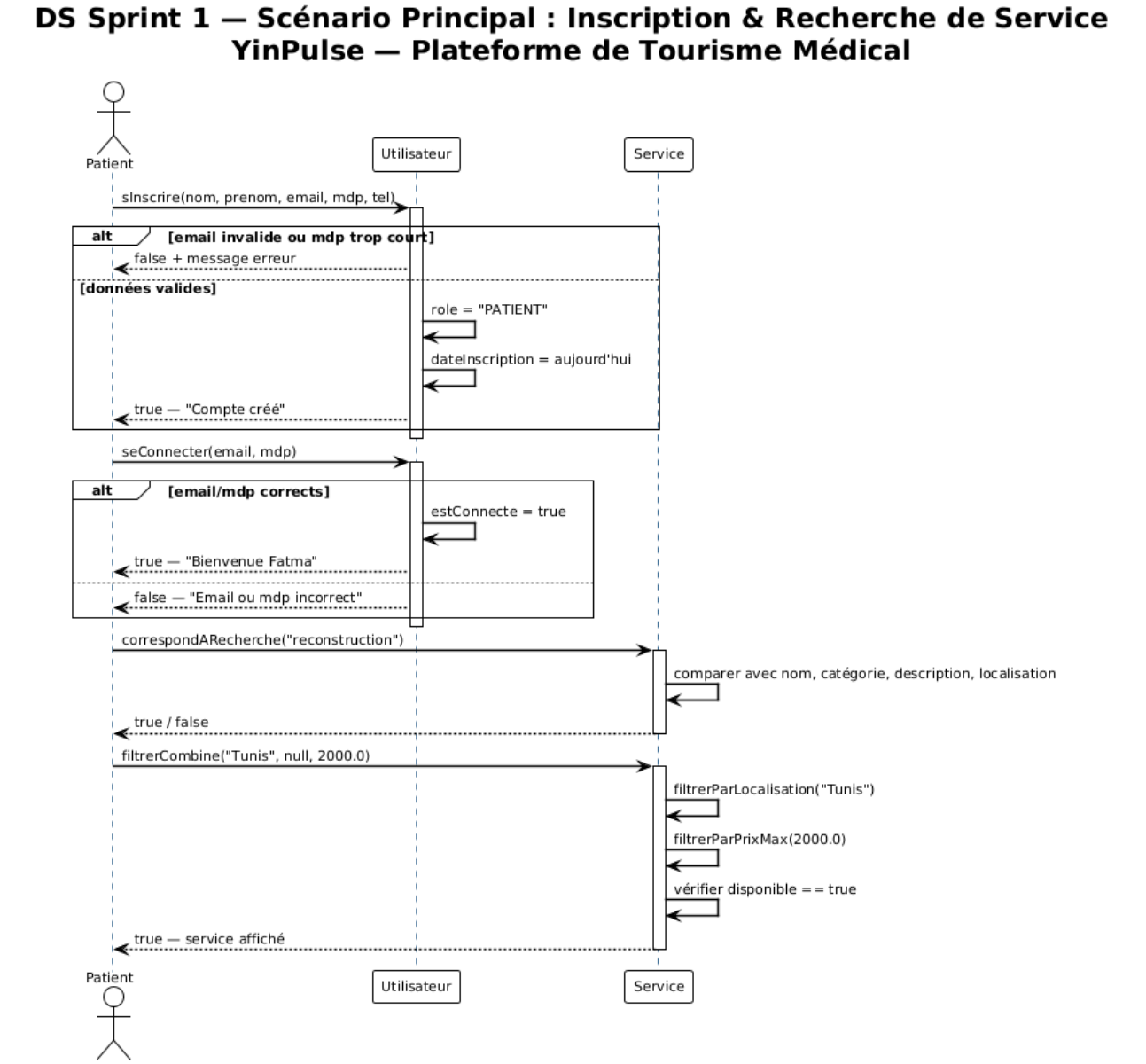
Classe	Responsable	Héritage / Relations	Méthodes métier clés
Utilisateur	Chaima	héritée par Admin	sInscrire(), seConnecter(), modifierMotDePasse()
Admin	Maram	extends Utilisateur (1 Admin gère 0..* Medecin)	afficherTableauDeBord(), ajouterMedecin(), validerAvis()
Service	Farah	géré par Admin (1 Service — 0..* Consultation)	correspondARecherche(), filtrerCombine(), mettreAJourNote()
Medecin	Malek	géré par Admin (1 Medecin — 0..* Consultation)	afficherProfil(), mettreAJourNote(), desactiver()
Avis	Eya	soumis par Utilisateur reçu par Medecin	soumettre(), valider(), rejeter(), repondre()
Consultation	Commune	Utilisateur 1→0..* | Medecin 1→0..* Consultation 1→0..1 Avis	confirmer(), terminer(), peutEtreNotee()
Retraite	Farah	—	calculerPrixSejour(), recommanderPackage(), reserver()
Hotel	Malek	—	filtrerCombine(), reserver(), estAdaptePostOperatoire()
Parcours	Chaima	agrège Consultation, Retraite, Hotel	avancerEtape(), ajouterCout(), calculerProgression()

Choix de conception : L'héritage **Admin extends Utilisateur** applique le principe DRY — l'authentification est réutilisée sans duplication. La classe **Consultation** est commune (partagée entre toutes les fonctionnalités) pour éviter le couplage fort. Les classes **Retraite**, **Hotel** et **Parcours** ont été ajoutées au Sprint 2 pour modéliser le parcours complet 'Your Journey'.



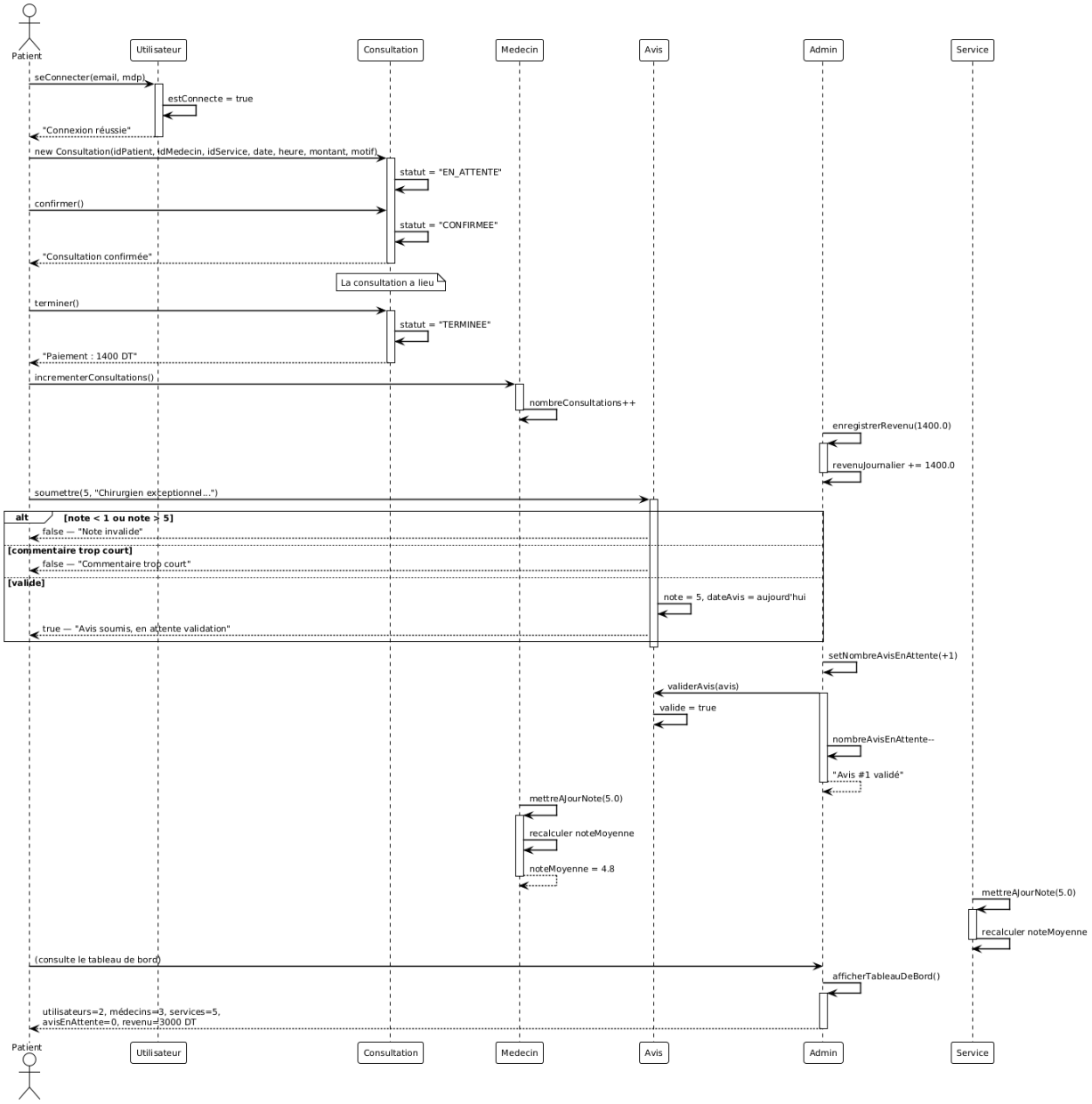
4.3 Diagrammes de Séquence

DS Sprint 1 — Scénario principal : Inscription & Recherche de Service. Acteurs : Patient, Utilisateur, Service. Fragments alt pour les cas d'erreur. Fichier : **diagrammes/DS_YinPulse_Sprint1.puml**



DS Sprint 2 — Scénario complet : Réservation → Consultation → Notation → Admin. Acteurs : Patient, Consultation, Medecin, Avis, Admin, Service, Parcours. Montre l'interaction entre toutes les fonctionnalités y compris le parcours 7 étapes. Fichier : **diagrammes/DS_YinPulse_Sprint2.puml**

DS Sprint 2 — Scénario Complet : Réservation → Consultation → Notation → Admin
YinPulse — Plateforme de Tourisme Médical



Lien Figma : (https://www.figma.com/make/gvJLKPRqJPXKhwzvkAdjWZ/Design-Yinflow-App-Branding?fullscreen=1&t=2PFwEmOx8Zi4NI7J-1) (prototype navigable). Captures dans docs/captures/maquettes/.

Chaque membre a conçu les écrans de sa fonctionnalité avec navigation cliquable entre écrans.

Membre	Fonctionnalité	Écrans conçus
Chaima	Authentification	Écran inscription (email, mdp, téléphone, validation temps réel) + Écran connexion
Farah	Recherche & Filtrage	Écran recherche (barre + filtres catégorie/localisation/budget) + Écran résultats (liste services, prix DT)
Malek	Profil Médecin	Fiche médecin complète (spécialité, tarif, note /5) + Écran modification profil (champs éditables)
Eya	Avis & Notation	Formulaire avis (note 1-5 étoiles + commentaire min 10 car.) + Liste avis validés (avec réponses admin)
Maram	Tableau de Bord	Dashboard admin (statistiques, alertes avis) + Écran gestion médecins (ajouter / désactiver)

Parcours utilisateur principal validé

1	**Page d'accueil**	Recherche par mot-clé ou catégorie depuis l'écran principal
2	**Résultats filtrés**	Clic sur un service → Profil médecin complet affiché
3	**Inscription / Connexion**	Création de compte Patient avec validation email + mdp
4	**Prise de RDV**	Consultation vidéo gratuite — début du parcours Your Journey
5	**Confirmation**	Parcours 7 étapes affiché avec progression et coût estimé
6	**Post-parcours**	Soumission d'avis (note 1-5 + commentaire)
7	**Validation Admin**	Admin valide l'avis → note médecin mise à jour

6. Code — Approche Hybride AGL + IA

Pour chaque fonctionnalité, l'approche en 3 étapes a été appliquée : (1) squelette AGL depuis le DC, (2) implémentation IA assistée, (3) relecture et corrections.

Chaima — Utilisateur.java | Product Owner + Dev | US-01, US-02

Prompt IA utilisé

```
Génère la méthode sInscrire() en Java pour une classe Utilisateur. Elle doit valider que
l'email contient @ et que le mot de passe a au moins 6 caractères. Afficher [OK] ou [ERREUR]
en français. Affecter le rôle PATIENT si l'inscription réussit.
```

Code final après corrections

```
public boolean sInscrire(String nom, String prenom, String email,
String motDePasse, String telephone) {
if (email == null || email.isEmpty() || !email.contains("@")) {
System.out.println(" [ERREUR] Email invalide.");
return false;
}
if (motDePasse == null || motDePasse.length() < 6) {
System.out.println(" [ERREUR] Mot de passe trop court.");
return false;
}
this.role = "PATIENT"; // correction IA : était absent
System.out.println(" [OK] Compte créé : " + prenom + " " + nom);
return true;
}
```

Correction apportée

L'IA avait oublié d'affecter this.role = "PATIENT". Ajout du null check sur l'email avant .contains().

Farah — Service.java | Scrum Master + Dev | US-03, US-04

Prompt IA utilisé

```
Génère filtrerCombine() pour une classe Service Java. Paramètres : localisation (String),
categorie (String), prixMax (double). Si un paramètre est null, il est ignoré. Utiliser les
méthodes existantes. Retourner true si le service est disponible et correspond à tous les
critères non-null.
```

Code final après corrections

```
public boolean filtrerCombine(String localisation, String categorie, double prixMax) {
boolean locOk = localisation == null || filtrerParLocalisation(localisation);
boolean catOk = categorie == null || filtrerParCategorie(categorie);
boolean prixOk = prixMax <= 0 || filtrerParPrixMax(prixMax); // bug fix
return locOk && catOk && prixOk && this.disponible;
}
```

Correction apportée

■■ L'IA avait généré prixMax == 0 au lieu de prixMax <= 0. Un budget de 0 DT aurait été ignoré plutôt que de bloquer tous les résultats.

Malek — Medecin.java | Développeur | US-05, US-06

Prompt IA utilisé

```
Génère mettreAJourNote() pour une classe Medecin Java. Calculer la note moyenne pondérée.
Arrondir à 1 décimale. Gérer le cas où nombreConsultations == 0 (premier avis).
```

Code final après corrections

```
public void mettreAJourNote(double note) {
if (nombreConsultations == 0) {
this.noteMoyenne = note; // ajout : cas division par zéro
} else {
double total = this.noteMoyenne * nombreConsultations;
this.noteMoyenne = Math.round(
((total + note) / (nombreConsultations + 1)) * 10.0) / 10.0;
}
}
```

Correction apportée

■■ L'IA n'avait pas géré le cas nombreConsultations == 0 (NaN au premier avis). Ajout de la branche conditionnelle pour éviter la division par zéro.

Eya — Avis.java | Développeur | US-07, US-08

Prompt IA utilisé

```
Génère soumettre() pour une classe Avis Java. Valider note entre 1 et 5. Valider commentaire
minimum 10 caractères. Afficher [OK] ou [ERREUR].
```

Code final après corrections

```
public boolean soumettre(int note, String commentaire) {
if (note < 1 || note > 5) {
System.out.println(" [ERREUR] Note invalide.");
return false;
}
if (commentaire == null || commentaire.trim().length() < 10) { // trim() ajouté
System.out.println(" [ERREUR] Commentaire trop court.");
return false;
}
this.commentaire = commentaire.trim();
System.out.println(" [OK] Avis soumis : " + note + "/5 - en attente.");
```

```
return true;
}
```

Correction apportée

■■ Correction critique : l'IA avait omis trim(). Sans ça, ' ' (10 espaces) aurait une longueur de 10 et passerait la validation — vulnérabilité de validation classique.

Maram — Admin.java | Développeur | US-09, US-10

Prompt IA utilisé

```
Génère la classe Admin en Java qui hérite de Utilisateur. Elle doit avoir un tableau de bord
(nombre utilisateurs, médecins, services, avis en attente, revenus du jour). Méthodes
ajouterMedecin(), validerAvis().
```

Code final après corrections

```
// Correction 1 : super() avec paramètres
public Admin(int id, String nom, String prenom, String email,
String motDePasse, String telephone, int niveauAcces) {
super(id, nom, prenom, email, motDePasse, telephone, "ADMIN", "2026-04-17");
// IA avait écrit : super(); — ne propageait pas les paramètres
}
// Correction 2 : guard avant décrémentation
public void validerAvis(Avis avis) {
avis.valider();
if (this.nombreAvisEnAttente > 0) this.nombreAvisEnAttente--; // ajouté
}
```

Correction apportée

■■ Correction 1 : super() vide ne propageait pas les 8 paramètres — Admin n'avait ni nom ni email.
Correction 2 : guard ajouté pour empêcher le compteur de descendre sous 0.

7. Tableau Comparatif AGL vs IA

Basé sur notre expérience réelle des 2 Sprints. 7 critères comparatifs issus des TD et de la pratique du projet.

Critère	AGL pur (Sprint 1 — squelette)	IA pure	Hybride (notre approche)
Cohérence structurelle	■ Garantie par le DC (toutes les classes correspondent aux diagrammes)	■ Variable selon le prompt (peut inventer des méthodes non prévues)	■ DC structure → IA implémente dans le cadre défini

Rapidité d'implémentation	■ Lente (méthodes vides à remplir manuellement)	■ Très rapide (code généré en secondes)	■ Rapide et structuré (squelette + IA = meilleur des deux)
Reproductibilité	■ Déterministe (même DC → même squelette)	■ Probabiliste (deux prompts identiques = résultats différents)	■ Déterministe sur la structure IA canalisée par le squelette
Qualité du code métier	■ Aucune logique (méthodes vides)	■■ À vérifier (3 bugs détectés : trim(), super(), prixMax)	■ Vérifié par review (3 bugs corrigés sur 5 classes)
Détection de bugs	■ Structure sans bug (aucune logique = aucun bug logique)	■ Bugs non détectés (IA ne teste pas son propre code)	■ Revue croisée via PRs (Farah a approuvé les 5 PRs)
Apprentissage équipe	■ Fort (comprendre le DC avant de coder)	■ Faible (risque de copier sans comprendre)	■ Fort (on corrige l'IA → on comprend vraiment)
Conformité au cahier des charges	■ Totale (squelette = traduction du DC)	■■ Partielle (IA peut ignorer des contraintes métier)	■ Totale (squelette force la conformité)

Conclusion : L'approche hybride est systématiquement supérieure aux deux approches pures. Le squelette AGL canalise l'IA dans des contraintes architecturales précises, ce qui réduit les hallucinations et garantit la cohérence avec les diagrammes UML.

8. Sprint Reviews & Rétrospectives

SPRINT 1 REVIEW

Le PO (Chaima) a validé : Les 5 classes compilent et s'exécutent. Main.java V1 produit un affichage lisible pour les 5 fonctionnalités. Les PRs sont mergées avec reviews. Maquettes Figma présentées et validées.

User Story	Statut	Commentaire
US-01 — Inscription	■ Done	Validation email + mdp fonctionnelle
US-02 — Connexion	■ Done	Session gérée correctement (estConnecte)
US-03 — Recherche	■ Done	Insensible à la casse, 4 champs couverts
US-04 — Filtrage	■ Done	Bug prixMax corrigé en review (== → <=)
US-05 — Profil médecin	■ Done	Affichage complet avec note et consultations
US-06 — Mise à jour	■ Done	Paramètres null ignorés correctement
US-07 — Soumettre avis	■ Done	trim() ajouté en review — sécurité validation
US-08 — Valider avis	■ Done	Statut EN_ATTENTE → VALIDÉ fonctionnel

US-09 — Tableau de bord	■ Done	Scénario 7 complet avec alertes
US-10 — Gérer médecins	■ Done	ajouterMedecin() + guard null fonctionnels

SPRINT 1 RÉTROSPECTIVE — animée par Farah (Scrum Master)

■ Ce qui a bien fonctionné

- L'approche hybride AGL + IA très productive : squelette en 30 min, implémentation IA en 1h par membre
- Les Pull Requests ont permis de détecter 3 bugs critiques (trim(), prixMax, super())
- Division des tâches (1 membre = 1 classe = 1 branche) a évité les conflits Git
- Le diagramme de classes détaillé a servi de guide — zéro refactoring structurel nécessaire

■ Ce qui doit être amélioré

- Gestion des dates : dateInscription en dur ('2026-04-17') — à remplacer par LocalDate.now() en Sprint 3
- Pas de vraie base de données — toutes les données sont en mémoire (perdues à l'arrêt)
- Tests unitaires manquants — la logique de validation n'est pas couverte par des JUnit

■ Actions concrètes pour Sprint 2

- Ajouter 4 nouvelles classes (Retraite, Hotel, Consultation, Parcours)
- Implémenter 7 scénarios complets de bout en bout dans Main.java V2
- Documenter les prompts IA utilisés dans le rapport

SPRINT 2 REVIEW

Incrément Sprint 2 : Main.java V2 exécute 7 scénarios complets. 9 classes Java opérationnelles. Logique métier réelle (calculs, validations, états). Rapport consolidé livré.

Scénario	Contenu	Résultat
Scénario 1	Inscription + Connexion (Fatma Ben Salah)	■ [OK] + [ERREUR] email invalide
Scénario 2	Recherche 'reconstruction' + Filtrage Tunis <= 2000 DT	■ 1 résultat + 2 résultats
Scénario 3	Exploration 3 retraites + recommandation MER <= 100 DT	■ Hammamet recommandée
Scénario 4	Sélection hotel post-op Hammamet <= 100 DT/nuit	■ Dar YinPulse sélectionné
Scénario 5	Parcours 7 étapes complet + coût total	■ TERMINÉ, 100%, 2239 DT
Scénario 6	Avis 5/5 soumis → validé → notes mises à jour3	■ Dr. Helali 4.8/5
Scénario 7	Tableau de bord admin final	■ 2 users, 3 médecins, 2023 DT

SPRINT 2 RÉTROSPECTIVE

■ Ce qui a bien fonctionné

- Scénarios de bout en bout très démonstratifs : chaque fonctionnalité interagit avec les autres
- Le parcours 7 étapes (Your Journey) est l'USP différenciante du projet
- Workflow Git propre : 5 branches → 5 PRs → reviews → merge dans main
- L'approche hybride a confirmé sa supériorité sur les 2 Sprints

■ Ce qui doit être amélioré (Sprint 3 si continuité)

- Interface web (Spring Boot + Thymeleaf ou React) pour remplacer la console
- Base de données relationnelle (PostgreSQL + JPA/Hibernate)
- Authentification JWT + gestion de sessions sécurisée
- Tests unitaires JUnit pour couvrir la logique de validation

. Bilan Individuel

Chaima	Product Owner + Dev	Utilisateur.java + Parcours.java

J'ai coordonné l'équipe tout en développant deux classes. En tant que PO, j'ai rédigé et priorisé le backlog, validé les incréments à chaque fin de sprint, et géré le workflow GitHub (branches, PRs, merges). Ma principale difficulté a été de concilier le rôle organisationnel (PO) et le rôle technique (Dev). J'ai appris l'importance de définir un backlog précis avant de coder : les critères d'acceptation m'ont guidée pour savoir exactement quoi implémenter. Le parcours 7 étapes (Parcours.java) était la fonctionnalité la plus ambitieuse — modéliser un état (progression, coût cumulé) a été un vrai défi de conception objet.

Farah	Scrum Master + Dev	Service.java + Retraite.java

Mon rôle de Scrum Master m'a appris à animer les cérémonies Scrum et à résoudre les blocages (notamment la coordination des PRs). Sur le plan technique, la méthode filtrerCombine() avec paramètres null a été la plus intéressante à concevoir : l'idée que null = 'critère ignoré' est un pattern puissant et générique. J'ai aussi été reviewer principale sur les 5 PRs, ce qui m'a permis de comprendre le code de tous les membres — une vraie valeur ajoutée.

Malek	Développeur	Medecin.java + Hotel.java

Mes deux classes représentent les acteurs principaux de la plateforme. La difficulté principale était la méthode mettreAJourNote() : gérer le cas du premier avis sans division par zéro demandait une réflexion sur les cas limites que l'IA n'avait pas anticipée. J'ai appris l'importance de tester les cas edge (0 avis, 1 avis, N avis) avant de considérer une méthode comme terminée. L'approche hybride m'a convaincu : l'IA est un très bon accélérateur mais pas un remplaçant du raisonnement.

Eya	Développeur	Avis.java

La classe Avis semblait simple au premier abord, mais la gestion des états (EN_ATTENTE → VALIDÉ → REJETÉ) et la validation du commentaire avec trim() ont révélé des subtilités importantes. J'ai découvert un bug de validation critique que l'IA avait omis : sans trim(), des espaces blancs auraient contourné la vérification de longueur. Ce bug m'a appris que la validation des entrées utilisateur doit toujours nettoyer les données avant de les vérifier. J'ai aussi compris l'importance de la visibilité des méthodes en Java (afficherEtoiles() private car usage interne uniquement).

Maram	Développeur	Admin.java

Implémenter Admin extends Utilisateur m'a permis de comprendre concrètement l'héritage Java et le principe DRY. Le bug du constructeur super() était subtil : l'IA avait généré super() vide alors qu'il fallait propager les 8 paramètres — sans ça, l'Admin n'avait ni nom ni email. J'ai aussi appris l'importance de la gestion des invariants (compteur jamais négatif). Le tableau de bord final du Scénario 7 est la pièce qui récapitule tout le travail de l'équipe en un seul affichage.

Prérequis

Outil	Version minimale	Vérification

JDK (Java Development Kit)	Java 11+ (testé avec JDK 17 et 21)	`java -version`
Git	Git 2.x	`git --version`

Étape 1 — Cloner le dépôt

```
# Cloner le dépôt GitHub
git clone https://github.com/chaimaarfaoui-ui/aglpro.git
cd aglpro
```

Étape 2 — Compiler le projet

Cette commande compile toutes les classes du package `com.YinPulse.models` vers le répertoire `out/` avec encodage UTF-8 (requis pour les caractères accentués) :

```
# Créer le répertoire de sortie (si absent)
mkdir -p out
# Compiler
javac -encoding UTF-8 -d out src/com/YinPulse/models/*.java
```

Étape 3 — Exécuter la démo Sprint 2

Lance `Main.java` qui exécute les 7 scénarios complets et affiche le résultat en console :

```
java -cp out com.YinPulse.models.Main
```

Sortie console attendue (extrait)

```
  ================================================================
      YinPulse - Plateforme de Tourisme Medical & Bien-etre
      Restore. Rebuild. Renew.
  ================================================================
  [OK] Compte cree : Fatma Ben Salah (fatma@gmail.com)
  [OK] Connexion reussie ! Bienvenue Fatma Ben Salah
  [ERREUR] Email invalide.
  ----------------------------------------------------------------
  SCENARIO 5 - Your Journey - Parcours complet 7 etapes
  [OK] Etape 1 completee : Consultation privee (video)
  ... (7 etapes) ...
  [SUCCES] Parcours complet pour Fatma Ben Salah !
  Cout total : 2239.0 DT  |  Statut : TERMINE  |  100%
```

Workflow Git — Branches & Pull Requests

```
# Voir toutes les branches
git branch -a
# Branches attendues :
  main
  feature/authentification         (Chaima — ≥ 6 commits)
  feature/recherche-services       (Farah — ≥ 6 commits)
  feature/profil-medecin           (Malek — ≥ 6 commits)
  feature/notation-avis            (Eya — ≥ 6 commits)
  feature/admin-dashboard          (Maram — ≥ 6 commits)
```

Dépôt GitHub

https://github.com/chaimaarfaoui-ui/aglpro

Structure du dépôt : `src/com/YinPulse/models/` contient les 9 classes Java. Les diagrammes PlantUML sont dans `diagrammes/` et les captures (Kanban, PRs, maquettes) dans `docs/captures/`.